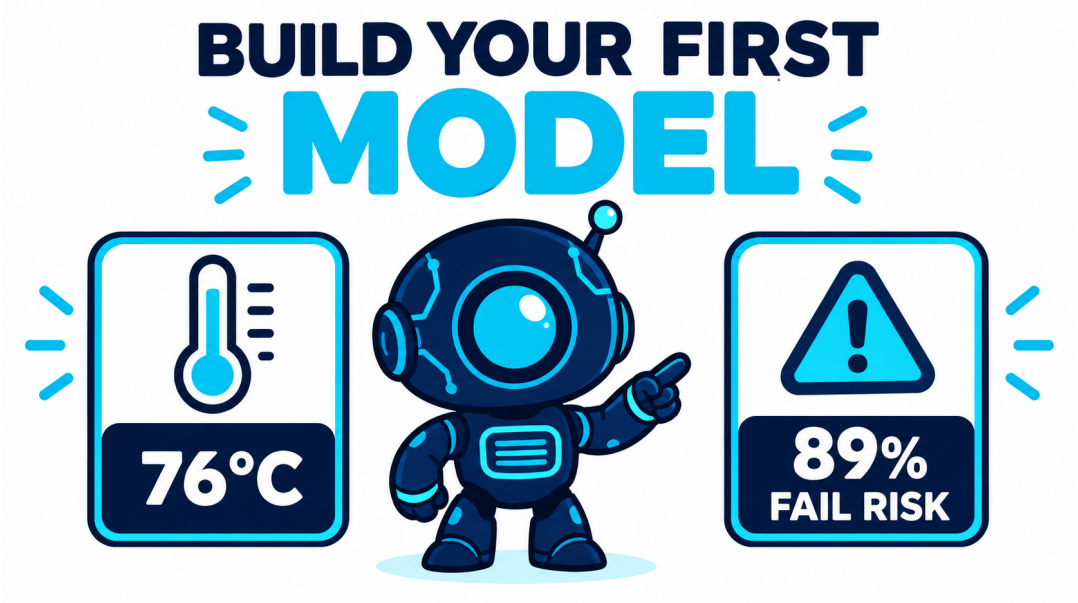
AI MECH LAB

AI Course — Beginner Track



Module 4, Topic 4.4

Predict-Outcome Capstone Project

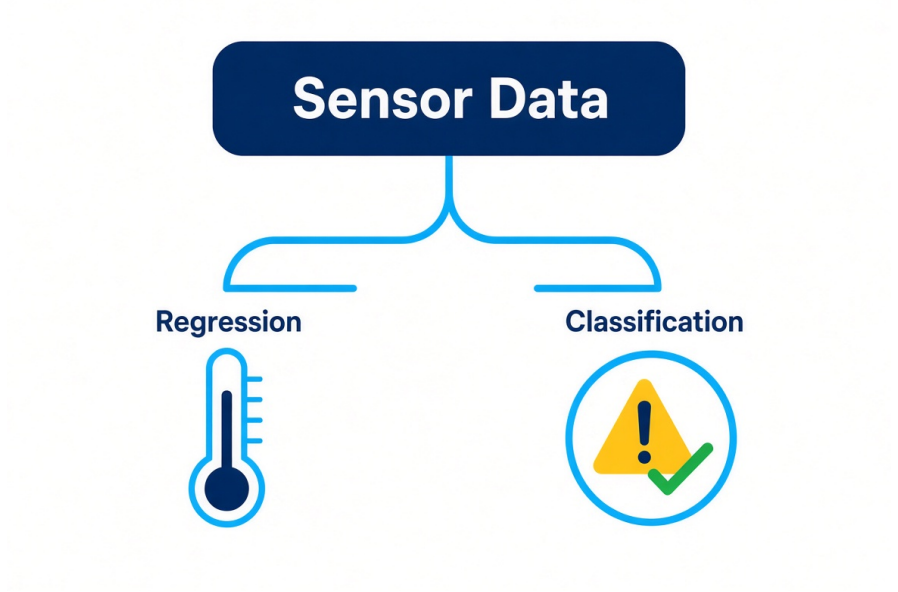
Lesson

You now have both tools of supervised learning: regression for predicting numbers, classification for predicting categories. This lesson puts them to work together in one applied project — end to end, from raw data to a working prediction. You won't just read about the steps this time. You'll walk through building something real.

The scenario

You're working with a small fleet of industrial motors, each fitted with basic sensors. For every motor, you have three readings — runtime hours since last service, average load percentage, and vibration level — plus two things you already know from maintenance records: the motor's current temperature, and whether it eventually failed within 30 days of that reading.

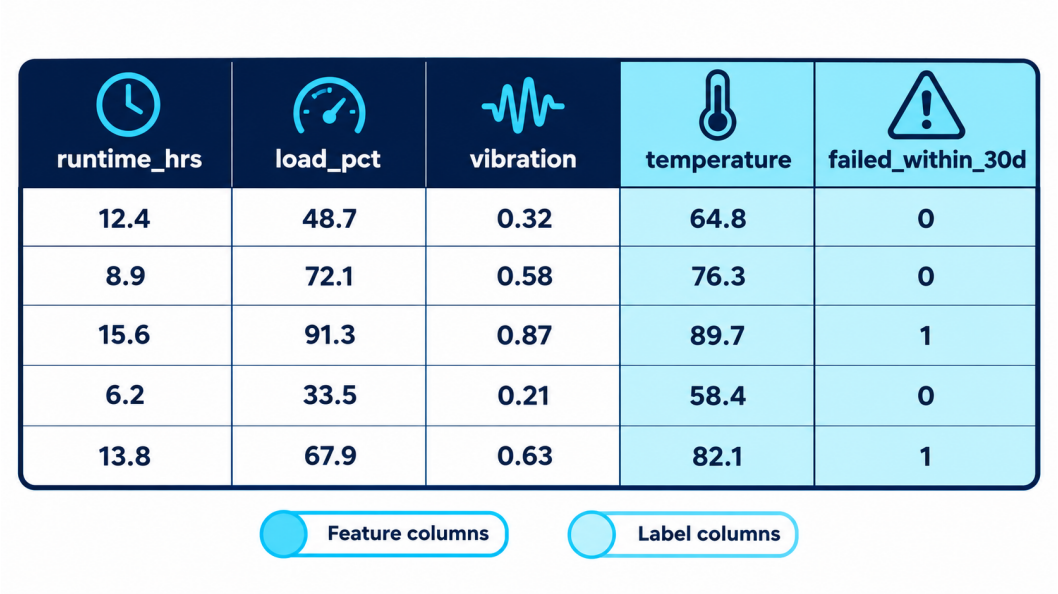
That gives you everything you need for two separate models: a regression model to predict temperature, and a classification model to predict fail/no-fail. Same data, two different kinds of questions.



One dataset, two different kinds of questions to answer.

Step 1 — Look at your data before you touch a model

Every real project starts here, not with code. Your dataset looks like this:



Three feature columns feeding two separate label columns.

Before building anything, ask what you'd ask of any dataset: which columns are features, and which is the label? Here, runtime_hrs, load_pct, and vibration are your features for both tasks. temperature is the label for the regression task. failed_within_30d is the label for the classification task. Same three inputs, two different outputs — that's the whole structure of this project.

Step 2 — Build the regression model: predicting temperature

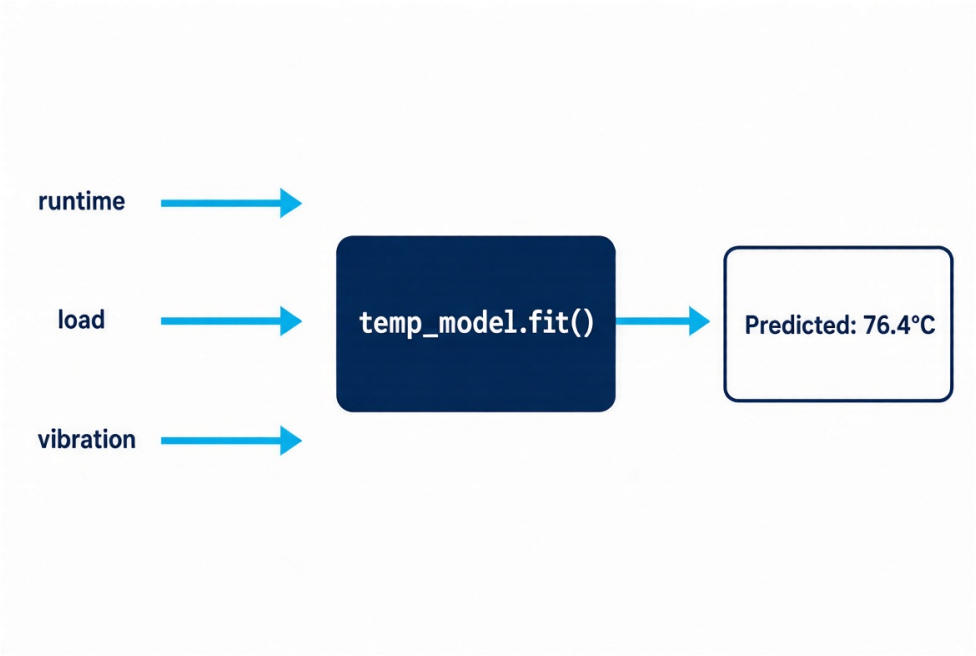
The goal here is exactly what you covered in Topic 4.2 — find the combination of weights across your three features that produces a predicted temperature as close as possible to the real value, minimizing error across the dataset.

```
from sklearn.linear_model import LinearRegression

X = df[["runtime_hrs", "load_pct", "vibration"]]
y_temp = df["temperature"]

temp_model = LinearRegression()
temp_model.fit(X, y_temp)

new_motor = [[600, 89, 6.0]]
predicted_temp = temp_model.predict(new_motor)
# predicted_temp ~ 76.4°C
```



Three features, one trained model, one predicted temperature.

Notice what's happening: fit() is the training step — the model searching for the best-fitting weights, same idea as the slope-and-intercept line from Topic 4.2, just now across three inputs instead of one. predict() is where you hand it a motor it hasn't seen and get a number back.

Step 3 — Build the classification model: predicting failure

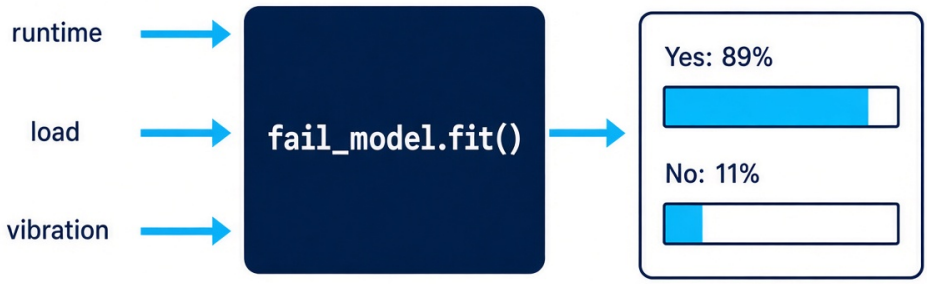
Same three features, different question. This time you're not predicting a number — you're predicting one of two categories: will this motor fail within 30 days, or not.

```
from sklearn.linear_model import LogisticRegression

y_fail = df["failed_within_30d"]

fail_model = LogisticRegression()
fail_model.fit(X, y_fail)

predicted_class = fail_model.predict(new_motor)
predicted_probability = fail_model.predict_proba(new_motor)
# predicted_class: "Yes"
# predicted_probability: [0.11, 0.89] -> 89% confident it fails
```

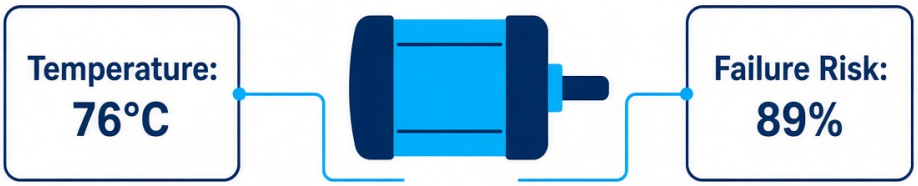


Same three features, a different model, a probability-based answer.

This is exactly the probability-based decision process from Topic 4.3 — the model isn't just saying "Yes," it's saying it's 89% confident, with the underlying probabilities available if you want them.

Step 4 — Put both predictions together

This is the payoff of doing both at once: for that same new motor, you now have two independent pieces of information from the same three sensor readings — a predicted temperature of roughly 76°C, and an 89% chance of failure within 30 days.

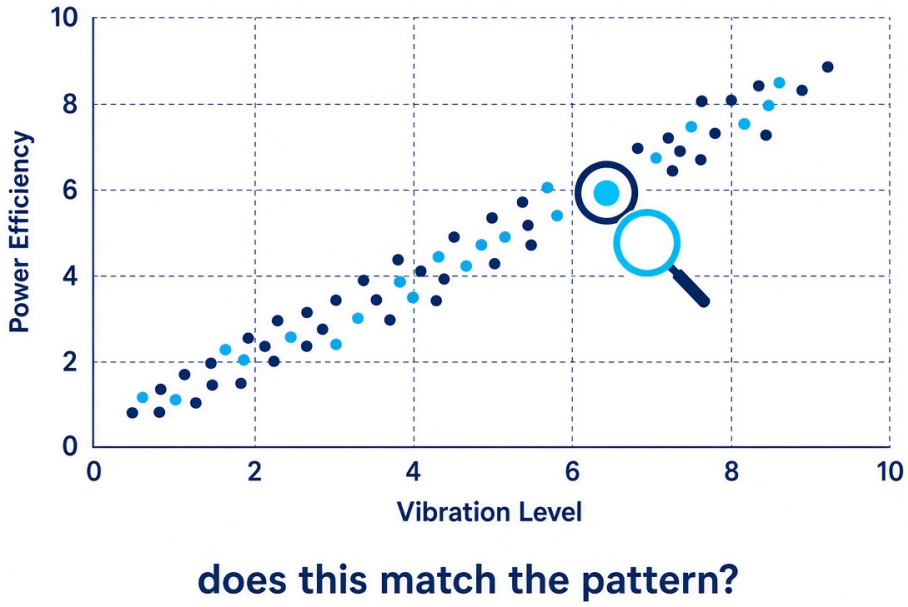


Two independent predictions, from one shared set of sensor readings.

Neither model knows about the other's output; each was trained on its own label. But used together, they give you a far more complete picture than either one alone: not just "will it fail," but a sense of the operating condition driving that risk.

Step 5 — Sanity-check the predictions, don't just trust them

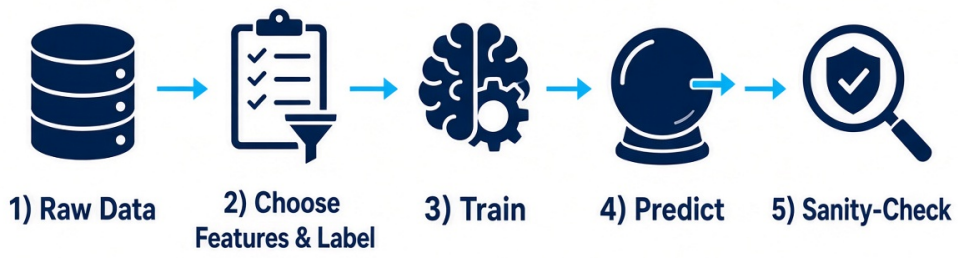
Before acting on either output, hold it up against what you already know. Does a predicted 76°C line up with the pattern in the training data, where high runtime and high load consistently pushed temperature into the 70s and 80s? Does an 89% failure probability line up with the fact that this motor's load and vibration numbers sit close to the two motors in your training data that actually did fail?



A new prediction checked against the pattern already visible in the data.

If a prediction looks wildly out of step with the patterns you can see by eye, that's a signal to check your data and your model before trusting the output — a habit worth building now, before the stakes get higher.

Where this leaves you



The complete loop: raw data, features and label, train, predict, sanity-check.

You've now taken raw sensor data all the way through to two working predictions, using the same three features to answer two different kinds of questions — one continuous, one categorical. That's the complete loop of supervised learning: choose your features, choose your label, train, predict, and sanity-check the result against what you understand about the system. This closes out Module 4 — and with it, the entire AI Beginner course.



Module 4 complete — the AI Beginner course is done.